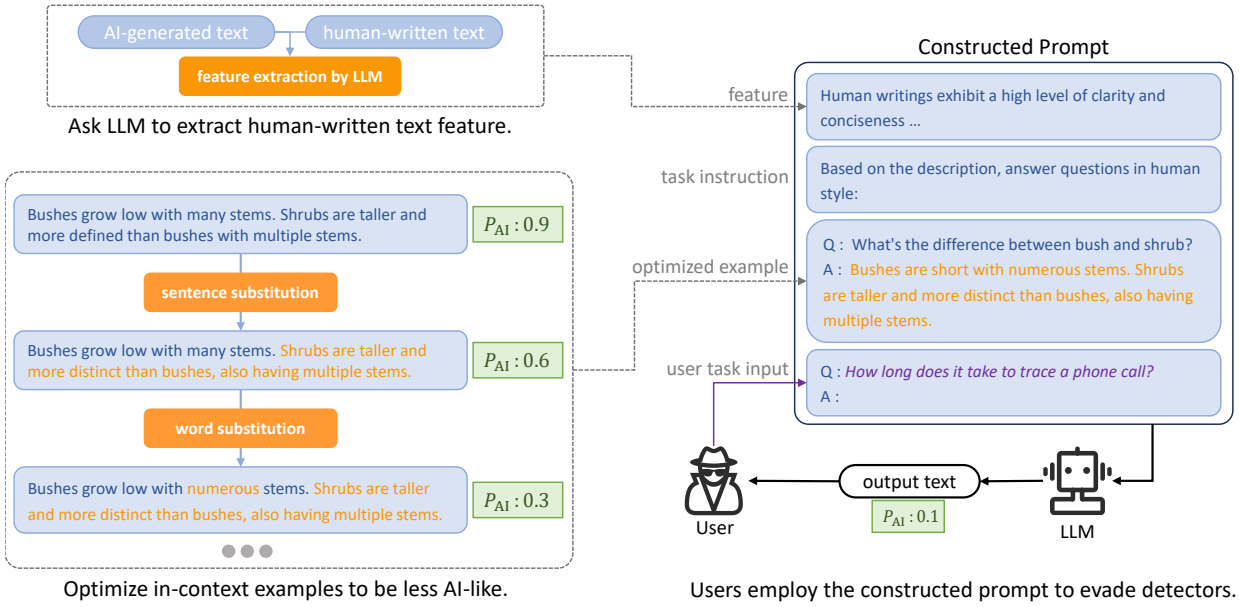

Figure 1: Illustration of how SICO generates prompts for the question answering task. The probability P_{AI}, as predicted by the proxy detector, indicates the likelihood that the given text is AI-generated. Once SICO prompt is constructed, it serves as a template, allowing users to insert various task inputs (highlighted in purple text).

prompt p, denoted by $\mathcal{U}(p)$, is defined as one minus the averaged predicted probability across X_{eval} (the higher $\mathcal{U}$, the better):

$$\mathcal{U}(p) = 1 - \frac{1}{|\mathbf{X}_{eval}|} \sum_{x \in X_{eval}} \mathcal{P}_{\text{AI}}(\text{LLM}(p \oplus x)). \tag{1}$$

3.2 Prompt Construction

Data collection We first collect a set of K triplets, i.e., $D = \{(x_{\text{ic}}^k, y_{\text{AI}}^k, y_{\text{human}}^k)\}_{k=1}^K$, where x_{ic}^k is a task input and $y_{\text{AI}}^k, y_{\text{human}}^k$ are the corresponding outputs generated by the LLM and humans, respectively. Note D is used for prompt construction and it is independent of X_{eval} which is used for prompt evaluation.

Feature extraction This step involves the K pairs of AI-generated and human-written outputs from D, denoted by $\{(y_{\text{AI}}^k, y_{\text{human}}^k)\}_{k=1}^K$. We provide LLM with these pairs and ask LLM to extract the distinct linguistic features of human-written text, denoted as t_{feature}. The top left of Figure 1 demonstrates this process and generate text to describe the feature of human-written text. This feature text is then utilized for sentence paraphrasing and included in the final prompt.

In-context example optimization We initialize the in-context examples as $(x_{\text{ic}}^k, y_{\text{ic}}^k)$, where y_{ic}^k is generated by paraphrasing y_{AI}^k. More specifically, the feature t_{feature} is concatenated with a paraphrasing instruction to instruct LLM to paraphrase the AI-generated text to obtain the initial y_{ic}^k. The paraphrasing instruction is "*Based on the description, paraphrase the following text to be human style*".

Then the in-context output y_{ic} is iteratively optimized to be less AI-like, as determined by the probability $\mathcal{P}_{\text{AI}}$ calculated by the proxy detector. By presenting more and more representative, i.e. lower $\mathcal{P}_{\text{AI}}$, in-context demonstrations to LLM, it is expected to understand how to generate human-like outputs. This in-context example optimization procedure is the key step in SICO for improving evasion performance, as verified by the ablation study in Section 5.1. Formally, the optimization goal can be expressed as:

$$y_{\text{ic}}^* = \underset{y'_{\text{ic}} \in \text{SIM}(y_{\text{ic}})}{\arg\min} \ \mathcal{P}_{\text{AI}}(y'_{\text{ic}}), \tag{2}$$

Algorithm 1 Substitution-based in-context example optimization (SICO)

Require: large language model LLM, prompt utility function $\mathcal{U}(\cdot)$, $D = \{(x_{\text{ic}}^k, y_{\text{AI}}^k, y_{\text{human}}^k)\}_{k=1}^K$, $\mathbf{X}_{eval}$, total iteration number N

1: Extract language feature t_{feature} using $\{(y_{\text{AI}}^k, y_{\text{human}}^k)\}_{k=1}^K$ and LLM
2: Construct in-context outputs $y_{\text{ic}}^k = \text{LLM}(t_{\text{feature}} \oplus p_{\text{para}} \oplus y_{\text{AI}}^k),\ \forall k \in \{1, ..., K\}$
3: Initialize $p^* \leftarrow t_{\text{feature}} \oplus p_{\text{task}} \oplus \{(x_{\text{ic}}^k, y_{\text{ic}}^k)\}_{k=1}^K$
4: **for** $n = 1$ to N **do**
5: **for** $k = 1$ to K **do**
6: Generate sentence-level / word-level substitutions C^k of y_{ic}^k, switching based on n
7: Optimize y_{ic}^k using Algorithm 2: $\hat{y}_{\text{ic}}^k \leftarrow \text{GreedyOPT}(y_{\text{ic}}^k, C^k)$
8: **end for**
9: Construct new prompt $\hat{p}$: $\hat{p} \leftarrow t_{\text{feature}} \oplus p_{\text{task}} \oplus \{(x_{\text{ic}}^k, \hat{y}_{\text{ic}}^k)\}_{k=1}^K$
10: **if** $\mathcal{U}(\hat{p}) > \mathcal{U}(p^*)$ **then**
11: Update in-context examples $y_{\text{ic}}^k \leftarrow \hat{y}_{\text{ic}}^k$ and update the best prompt $p^* \leftarrow \hat{p}$
12: **end if**
13: **end for**
14: **return** p^*

where $\text{SIM}(y_{\text{ic}})$ denotes the set of text that is semantically similar to y_{ic}. The goal of setting such semantic restriction is to maintain the usability of the text during optimization. In SICO, we generate semantically similar text by replacing words and rephrasing sentences. This is explained in detail below.

Substitution type To generate y'_{ic} that is semantically similar to y_{ic}, we employ substitution at word level and sentence level in turn. For word-level substitution, we use WordNet (Miller, 1998), a lexical database of English words, to construct a synonym substitution set. We restrict substitutions to content words[2] and ensure that the substitution would not change the part-of-speech tags. We use a mask language model to filter out the candidate words that not fits the context. For sentence-level substitution, we utilize the paraphrasing instruction combined with extracted feature, denoted as $t_{\text{feature}} \oplus p_{\text{para}}$. This combined instruction is used to prompt LLM to generate paraphrases for each y_{ic}. Using t_{feature} will make the generated paraphrases be more human-like, thus increasing the efficiency of optimization.

Algorithm As illustrated in Algorithm 1, SICO would optimize $\{y_{\text{ic}}^k\}_{k=1}^K$ for N iterations (lines 4-17). At each iteration, each y_{ic}^k would be optimized by greedy substitution (line 11), as presented in Algorithm 2. Specifically, for the i-th original word/sentence y_i in the text y, let $C_{i,j}$ denote its j-th synonym/paraphrase, and let $\text{SUB}(y_i, C_{i,j})$ denote the new text resulting from substituting y_i with $C_{i,j}$. For each substitution position i, SICO identifies the best substitution $C_{i,*}$ by checking which $C_{i,j}$ results in the lowest AI probability when replacing y_i (Line 1 in Algorithm 2).

Algorithm 2 Greedy text optimization (GreedyOPT)

Require: Text y, substitutions C of y, proxy detector $\mathcal{P}_{\text{AI}}$

1: $C_{i,*} = \arg\min_{C_{i,j}} \mathcal{P}_{\text{AI}}(y_{(i,j)}),\ \forall y_i \in y$
where $y_{(i,j)} = \text{SUB}(y_i, C_{i,j})$
2: **for** each y_i in y **do**
3: $y \leftarrow \text{SUB}(y_i, C_{i,*})$
4: **end for**
5: **return** y

After obtaining the optimized in-context output $\hat{y}_{\text{ic}}$, the new prompt is constructed as $\hat{p} = t_{\text{feature}} \oplus p_{\text{task}} \oplus \{(x_{\text{ic}}^k, \hat{y}_{\text{ic}}^k)\}_{k=1}^K$, where p_{task} is the task instruction, as illustrated in Figure 1. Then $\hat{p}$ would be compared with the current best prompt p^* based on their utility scores as defined in Eq. (1). If $\hat{p}$ scores higher, SICO replaces p^* with it. After N iterations, p^* is returned as the final prompt. More implementation details of SICO are shown in Appendix A.

[2]Content words are the words that carry meanings, consisting of nouns, verbs, adjectives and adverbs.

3.3 SICO for Paraphrasing

The approach described above directly generates the task output to evade detectors. We refer to this direct approach as SICO-Gen. Alternatively, SICO can be easily adapted for paraphrasing, which we term as SICO-Para. Instead of direct generation, SICO-Para evades detectors in two steps. Initially, LLM produces an intermediate task output, typically incapable of evading detectors. Then. this output is paraphrased using SICO-Para to successfully evade detectors. Switching from SICO-Gen to SICO-Para requires only two adjustments: (1) the task input x is set to the AI-generated output text in D and $\mathbf{X}_{eval}$; (2) task instruction p_{task} is modified to paraphrasing instruction.

4 Experiments

4.1 Experimental Setup

Tasks & datasets We consider three real-world tasks that are susceptible to the misuse of LLMs, i.e., academic essay writing (Writing), open-ended question answering (QA), and fake review generation (Review). We use GPT-3.5, one of the most powerful LLMs, to complete the tasks and generate text in our experiments.

For academic writing, we employ Wikipedia paragraphs from SQuAD dataset (Rajpurkar et al., 2016) as human-written text. Following the approach in Mitchell et al. (2023), we use the first 30 words of these paragraphs as task inputs and ask GPT-3.5 to complete the rest. For open-ended question answering, we sample questions from Eli5 (Fan et al., 2019) dataset and ask GPT-3.5 to generate answers, following Krishna et al. (2023). For fake review generation, we first instruct GPT-3.5 to extract the business name and five keywords from human-written reviews from Yelp dataset (Zhang et al., 2015), and then generate fake reviews based on the extracted information with specified sentiment. For each task, we collect 200 examples from GPT-3.5 (called original AI-generated text) and 200 human-written examples from corresponding dataset. More details about dataset can be found in Appendix E.

Detectors Six representative detectors belonging to three different types are considered. Details of these detectors can be found in Appendix C.

Training-based methods. (i) GPT-3.5 Detector (GPT3-D) (Guo et al., 2023): a RoBERTa model (Liu et al., 2019) fine-tuned on text generated by GPT-3.5. (ii) GPT2 Detector (GPT2-D) (Solaiman et al., 2019): a RoBERTa detector officially released by OpenAI, fine-tuned on GPT2-generated text.

Statistical methods. (i) DetectGPT (Mitchell et al., 2023) evaluates the variation in a language model's log probability by introducing minor perturbations to the detected text. (ii) Log-Rank (Mitchell et al., 2023) is a statistical method that employs a language model to compute the mean prediction rank of each token in a text, given its preceding context. We utilize a relatively smaller language model, GPT2-medium (Radford et al., 2019), for both methods. Because Mireshghallah et al. (2023) find that smaller language models have better detection performance than larger ones.

APIs.[3] (i) GPTzero (Tian, 2023) is a widely-used commercial detector, cooperated with many academic organizations. (ii) OpenAI Detector (OpenAI-D) (OpenAI, 2023) is officially offered by OpenAI, fine-tuned from a language model.

Baselines We consider four paraphrasing baselines that evade detectors by paraphrasing the original AI-generated text. Specifically, two recently proposed methods are considered: (1) *Parrot* (Sadasivan et al., 2023) and (2) *DIPPER* (Krishna et al., 2023). Both methods employ an external neural network specifically trained for paraphrasing. In addition, we include two prompting baselines to instruct GPT-3.5 to paraphrase the original AI-generated text: (3) *GPT-Para* that uses the straightforward instruction *"Paraphrase this"* to assess the capabilities of GPT-3.5 without intricate prompt engineering, and (4) *Human Prompt* that utilizes a human-designed prompt. More details can be found in Appendix A.2.

Evaluation metrics We use the area under the ROC curve (AUC) to measure the performance of detectors. The ROC curves are also illustrated to show the detection performance under different classification thresholds.

[3]We consider the API versions of May 15, 2023. For OpenAI-D, we follow the implementation of Krishna et al. (2023).